

★ WEEK 12 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Measure Meaning · Run the Softmax · Retrieve Like a RAG

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — this week's corpus is built inline; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Cosine	toy 2-D vectors on the slides	fml_w12_p1_vectors.csv (hand-calc)
P2 · Softmax	scores on the slides — pure pencil work	— (3 candidates, no file needed)
P3 · Tiny RAG	keyword-count vectors on the slides	fml_w12_p3_docs.csv (hand-calc)
Scaled labs	corpus defined in-script (no download)	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · COSINE

Measure Meaning With an Angle

Four toy sentence-fingerprints — dot products, norms, and a similarity ladder

Four Sentences, Four Fingerprints

Toy 2-D embeddings — compute every pairwise cosine with sentence A

Sentence	Fingerprint
A · 'The Fed raised interest rates'	(2, 1)
B · 'The central bank hiked rates'	(4, 2)
C · 'Refunds are allowed within 30 days'	(-1, 2)
D · 'Diversification lowers portfolio risk'	(0, 3)

Helper

- $\sqrt{5} \approx 2.236 \cdot \sqrt{20} \approx 4.472$.
- $\cos = \text{dot product} \div \text{product of lengths}$.

Your deliverables

- $\cos(A,B), \cos(A,C), \cos(A,D)$.
- A similarity ranking, with one-line readings.

The Tool: Direction Over Length

Two vectors can differ in size and still mean the same thing — the angle sees through it

$$\cos \theta = \frac{a \cdot b}{\|a\| \|b\|} \quad a = (2, 1) \quad b = (4, 2) \quad c = (-1, 2) \quad d = (0, 3)$$

Reading it

- The dot product rewards pointing the same way.
- Dividing by both lengths removes 'loudness' — only direction remains.
- $B = 2 \times A$ exactly: a longer arrow, the same meaning.

Lab guide — per pair

- Step 1: dot product $a \cdot b$.
- Step 2: lengths $|a|$ and $|b|$ (helpers given).
- Step 3: divide; place on the ladder $0 \rightarrow 1$.

Three Cosines, One Ladder

From perfect paraphrase to perpendicular stranger

Pair	Dot product	$ a \cdot b $	Cosine	Reading
A vs B	$2 \cdot 4 + 1 \cdot 2 = 10$	$2.236 \times 4.472 = 10$	1.00	paraphrases — same direction exactly
A vs D	$2 \cdot 0 + 1 \cdot 3 = 3$	$2.236 \times 3 \approx 6.71$	0.45	cousins — finance, different topic
A vs C	$2 \cdot (-1) + 1 \cdot 2 = 0$	$2.236 \times 2.236 = 5$	0.00	strangers — perpendicular meanings

$$\cos(a, b) = 1.0 \text{ (same direction)} \quad \cos(a, d) \approx 0.45 \quad \cos(a, c) = 0 \text{ (unrelated)}$$

The Angle Sees What Spelling Cannot

A and B share zero words beyond 'rates' — and score a perfect 1.0

1.00

A vs B — different words, identical direction

0.45

A vs D — same industry, different subject

0.00

A vs C — monetary policy vs refund policy

2×

B is exactly twice A: length ignored, meaning kept

Why angle, not distance

- Longer documents produce longer vectors — distance punishes them.
- Cosine compares direction only: a tweet can match a filing.

Where P3 goes

- Replace 'sentence vs sentence' with 'question vs documents'.
- argmax of this exact number IS the R in RAG.

Scale It: a Real Similarity Map of Finance Text

Paste the metaprompt — Claude Code embeds a corpus and draws the meaning map

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: vectors (2,1), (4,2), (-1,2),  
   (0,3) -> confirm  $\cos(A,B)=1.0$ ,  $\cos(A,D)=0.447$ ,  
    $\cos(A,C)=0$ ; (reference: fml_w12_p1_vectors.csv)  
2) defines a 15-sentence finance corpus in-script:  
   5 about rates/inflation, 5 about refunds/policies,  
   5 about earnings/results;  
3) embeds with TfidfVectorizer and computes the full  
   15 x 15 cosine matrix; plot it as a heatmap;  
4) prints the top-3 most similar pairs and the 3 most  
   dissimilar pairs with their sentences;  
5) three-sentence verdict: do the three topic blocks  
   appear in the heatmap, and what that says about  
   'meaning = location'.
```

[OUTPUT] What to expect

```
toy check: 1.0 / 0.447 / 0  
           exactly  
heatmap: three bright  
         blocks on the diagonal  
top pairs: within-topic  
bottom pairs: across-topic  
  
Your 2-D ladder, grown to  
a 15-sentence map.
```

P2

PROBLEM 2 · SOFTMAX

One Next-Word Choice, Fully by Hand

Three candidates, three temperatures — the LLM's dice, exposed

'Yesterday the market ____'

The model scores three candidates: $z = (\text{rose}: 2, \text{fell}: 1, \text{banana}: -1)$. Turn scores into probabilities at three temperatures

Candidate	rose	fell	banana
Score z	2	1	-1

Helpers (e-values)

- $e^2 \approx 7.389 \cdot e^1 \approx 2.718 \cdot e^1 \approx 0.368$.
- $e^4 \approx 54.60 \cdot e^{0.5} \approx 1.649 \cdot e^{-0.5} \approx 0.607 \cdot e^{-2} \approx 0.135$.
- Temperature divides every score BEFORE the exponent.

Your deliverables

- Probabilities at $T = 1, T = 0.5, T = 2$.
- What each temperature does to 'banana'.
- The right T for a compliance chatbot, in one line.

The Tool: Softmax — Scores Into a Betting Slip

Exponentiate, then normalize; temperature reshapes the slip without new knowledge

$$P(w) = \frac{e^{zw/T}}{\sum_v e^{zv/T}} \quad z = (\text{rose: } 2, \text{ fell: } 1, \text{ banana: } -1)$$

Reading it

- e^z makes every score positive and stretches the gaps.
- Dividing by the sum turns them into probabilities.
- T rescales the gaps first: small T stretches, large T squashes.

Lab guide — per temperature

- Step 1: divide each z by T .
- Step 2: exponentiate with the helpers.
- Step 3: sum, divide, read the slip.

Nine Exponentials, Three Betting Slips

Same knowledge every time — only the nerve changes

	T = 1 (neutral)	T = 0.5 (cold)	T = 2 (hot)
z / T	(2, 1, -1)	(4, 2, -2)	(1, 0.5, -0.5)
$e^{(z/T)}$	7.389 / 2.718 / 0.368	54.60 / 7.389 / 0.135	2.718 / 1.649 / 0.607
Sum	10.48	62.12	4.97
P(rose / fell / banana)	70.5 / 25.9 / 3.5 %	87.9 / 11.9 / 0.2 %	54.7 / 33.1 / 12.2 %

$T=1$: 70.5/25.9/3.5 % $T=0.5$: 87.9/11.9/0.2 % $T=2$: 54.7/33.1/12.2 %

Temperature Is Nerve, Not Knowledge

The scores never changed — 'banana' went from 1-in-500 to 1-in-8

3.5%

Banana at $T = 1$ — the raw learned odds of nonsense

0.2%

Banana at $T = 0.5$ — cold sampling nearly kills it

12.2%

Banana at $T = 2$ — hot sampling feeds it, $3.5\times$ up

$\times 60$

Banana's odds ratio between cold and hot — one dial

The chatbot setting

- Compliance, numbers, policies: run cold ($T \leq 0.5$).
- Brainstorming taglines: warm is a feature, not a bug.

The honest limit

- Cold sampling reduces randomness, not wrongness.
- A confidently wrong top choice stays wrong — that cure is P3's.

Scale It: Sample the Slip a Thousand Times

Paste the metaprompt — Claude Code turns the arithmetic into behavior

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) implements softmax(z, T) in numpy; verify the toy:  
   z = (2, 1, -1) -> T=1: (70.5, 25.9, 3.5)%,  
   T=0.5: (87.9, 11.9, 0.2)%, T=2: (54.7, 33.1, 12.2)%;  
2) plots the three distributions as grouped bars;  
3) samples 1000 'next words' at each T with the given  
   probabilities; prints how many times banana appears  
   at each temperature;  
4) sweeps T from 0.1 to 3 and plots P(banana) vs T;  
5) generates two 12-word sentences from a tiny 3-gram  
   toy model of finance headlines, one at T=0.3 and  
   one at T=2, and prints both;  
6) three-sentence verdict: what T buys and what it can  
   never fix, in this week's language.
```

[OUTPUT] What to expect

```
toy check: all three slips  
exact  
banana count: ~2 (cold),  
              ~35 (neutral), ~120 (hot)  
P(banana) curve: rises  
                 smoothly with T  
cold sentence: dull, safe  
hot sentence: colorful,  
              occasionally absurd  
  
Your nine exponentials,  
rolled 3,000 times.
```

P3

PROBLEM 3 · TINY RAG

Retrieve, Then Ground

One question, three documents — the R in RAG, by hand

A Customer Asks the Chatbot

Q: 'How long do I have to return something?' Vocabulary: (rates, inflation, refund, return, earnings) — $q = (0, 0, 0, 1, 0)$

Document	Keyword-count vector
d_1 · 'The Fed raised rates to fight inflation'	(1, 1, 0, 0, 0)
d_2 · 'Refund policy allows returns within 30 days'	(0, 0, 1, 1, 0)
d_3 · 'Quarterly earnings beat expectations'	(0, 0, 0, 0, 1)

Helper

- $|q| = 1 \cdot |d_1| = |d_2| = \sqrt{2} \approx 1.414 \cdot |d_3| = 1.$
- $1/\sqrt{2} \approx 0.707.$

Your deliverables

- $\cos(q, d)$ for all three documents.
- The retrieved doc + the grounded prompt.

The Tool: argmax of Cosine, Then a Fence

Retrieval picks the page; the prompt builds the fence around the LLM

retrieved doc = $\arg \max_d \cos(q, d)$ then: “answer ONLY from the retrieved text”

Reading it

- The question becomes a vector in the SAME space as the documents.
- Cosine ranks every document against it; the top one is retrieved.
- P1's arithmetic, wearing a librarian's badge.

Lab guide

- Step 1: dot q with each document vector.
- Step 2: divide by the norms (helpers given).
- Step 3: retrieve the argmax; write the fenced prompt.

Three Cosines and One Fence

Only d_2 shares a direction with the question — retrieval is that simple

Document	$q \cdot d$	$ q \cdot d $	Cosine	Verdict
d_1 (rates, inflation)	0	1×1.414	0	ignore
d_2 (refund, return)	1	1×1.414	0.707	RETRIEVE ✓
d_3 (earnings)	0	1×1	0	ignore

The grounded prompt (write it out)

- "Context: 'Refund policy allows returns within 30 days.' Answer ONLY from the context above and cite it. If the answer is not in the context, say NOT FOUND."

The Guessing Machine Became a Reading Machine

Retrieval chose the page; the fence made the LLM stay on it

0.707

d_2 's cosine — the only document sharing the question's direction

30 days

The grounded answer
— read, not guessed

2 zeros

d_1 and d_3 — cleanly
filtered before the LLM
ever spoke

1 fence

'ONLY from the
context' + a NOT-
FOUND valve

What made it safe

- Wrong-topic documents scored exactly 0 — no accidental grounding.
- The valve handles the silent-corpus case: refusal beats invention.

Scaling up

- Real systems: neural embeddings, thousands of chunks, top-k rerank.
- Same argmax, same fence — the hand-calc IS the architecture.

Scale It: a Working Mini-RAG With an Honesty Valve

Paste the metaprompt — Claude Code assembles retrieval, fence, and refusal

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first:  $q = (0,0,0,1,0)$  against the  
   three count vectors -> confirm cosines 0 / 0.707 / 0;  
   (reference: fml_w12_p3_docs.csv)  
2) builds a 12-document finance corpus in-script  
   (policies, rates, earnings) and a TF-IDF retriever;  
3) runs 6 test questions; for each, print the top doc,  
   its score, and the assembled grounded prompt with  
   the fence: 'Answer ONLY from the context; if absent,  
   say NOT FOUND';  
4) includes 2 questions whose answers are NOT in the  
   corpus; apply a 0.15 threshold and show the valve  
   returning NOT FOUND;  
5) three-sentence verdict: which failures the  
   threshold prevented, in this week's language.
```

[OUTPUT] What to expect

```
toy check: 0 / 0.707 / 0  
          exactly  
6 retrievals: right topic  
              each time, scores shown  
2 off-corpus questions:  
  NOT FOUND (the valve)  
prompts printed ready to  
paste into any LLM
```

Your one-question RAG,
grown to a working desk.

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1**Cosine ladder: 1.00 / 0.45 / 0.00**

Paraphrase, cousin, stranger — the angle saw through spelling, and $B = 2A$ proved length is ignored.

2**Softmax slips: 70.5 $\rightarrow$ 87.9 $\rightarrow$ 54.7% on 'rose'**

Temperature is nerve, not knowledge: banana went $\times 60$ between cold and hot on unchanged scores.

3**Retrieval: $\cos 0.707$ picked d_2 , the fence kept the LLM on it**

$\text{argmax} + \text{'ONLY from the context'} + \text{a NOT-FOUND valve}$ — the hand-calc IS the RAG architecture.

Hand calculation first — then let Claude Code scale the same idea to real data.